Problem 2. Final Exam

Input file:	*standard input*
Output file:	*standard output*
Time limit:	12 seconds
Memory limit:	256 mebibytes

Rikka is a talented student.

She spends almost every day on ICPC. But the final exam is approaching.

Rikka plans to grasp- the last minute to review the courses before the exam. She has up to M minutes for review and then takes n consecutive exams. If Rikka spends x minutes on the review for the i-th exam, she would get $f_i(x)$ points, where $f_i(x) = \max\{0, \min\{d_i, a_i x^2 + b_i x + c_i\}\}$ with the exam-specific parameters a_i, b_i, c_i, d_i.

Rikka wants to maximize the total score of her n exams. Note the minutes she spends in reviewing a certain course can be any non-negative real number. Also, she does not have to spend all of her M minutes on the review so that she can spend more time on ICPC.

Input

The first line contains an integer n and a real number M.

Each of the following n lines contains four real numbers a_i, b_i, c_i, d_i, denoting the parameters of all the n exams.

It is guaranteed that $1 \le n \le 100\,000$, $0 < M \le 10^8$, $|a_i| \le 10$, $|b_i| \le 5000$, $0 \le c_i \le d_i \le 5000$, and all real numbers in the input are given with exactly three decimal places.

It is guaranteed that there are at most 18 exams with $a_i > 0$.

Output

You need to output d, the maximum total score that Rikka can get. Assuming the correct result is d^*, you need to ensure that $\frac{|d-d^*|}{\max\{d^*,1\}} \le 10^{-6}$.

Example

standard input	standard output
4 2.000 0.000 7.000 3.000 10.000 -1.000 10.000 3.000 10.000 -2.000 10.000 3.000 10.000 -3.000 10.000 3.000 10.000	29.5734198185

Problem 3. Revenue

Input file:	*standard input*
Output file:	*standard output*
Time limit:	1 second
Memory limit:	256 mebibytes

There is a seller who has n items for sale to a single buyer. The buyer has a *valuation profile* $\bar{v} = (v_1, \ldots, v_n)$, where $v_j \geq 0$ denotes her value for item j.

The seller can set a *pricing* $\bar{p}$, that is, a vector of item prices $(p_1, \ldots, p_n)$. Given a pricing $\bar{p}$, the *utility* of buying item j is $v_j - p_j$. The buyer will purchase a single item j that maximizes her utility, or nothing if her utility from purchasing any item would be negative. If there are multiple items with the same maximal utility, she will choose the one with the minimal price. The *revenue* of the seller is defined as the price of the item that the buyer buys, and if the buyer buys nothing, the revenue is 0.

Now we know that the valuation profile $\bar{v}$ is drawn from a *joint distribution* F which defines the probability of every possible value of $\bar{v}$. Unfortunately, we **do not** know F. Instead, we know the *marginal distributions* $F_1, F_2, \ldots, F_n$: distribution F_i defines the probability of $v_i = x$ for every possible x. But we do not know how they are correlated: the values are not necessarily independent, so the individual probabilities of $v_i = x$ and $v_j = y$ don't define the probability of both happening simultaneously. Note that the joint distribution F is over the valuation profile $\bar{v}$ and that the marginal distribution F_i is over the value v_i of item i.

Given the pricing $\bar{p}$ and the marginal distributions $F_1, F_2, \ldots, F_n$, we are now asked to compute the minimal expected revenue among all possible joint distributions. Formally, let $\mathcal{F}$ be the set of joint distributions over valuation profiles $\bar{v}$ whose marginal distributions for the individual item values are just $F_1, F_2, \ldots, F_n$. Let $\mathrm{Rev}(\bar{p}, F)$ be the seller's expected revenue from setting a pricing $\bar{p}$, if the valuation profile $\bar{v}$ is drawn from a joint distribution F. We are asked to compute

$$\min_{F \in \mathcal{F}} \mathrm{Rev}(\bar{p}, F).$$

Input

The first line contains a single integer n ($1 \leq n \leq 10^5$), the number of items for sale.

The second line contains n non-negative integers $p_1, p_2, \ldots, p_n$ ($0 \leq p_i \leq 10^5$), the pricing vector $\bar{p}$.

Next n lines describe the marginal distributions $F_1, F_2, \ldots, F_n$. Each line starts with an integer k: the support size (number of different values) of F_i. Then follow k pairs of numbers q_j and v_j ($0 \leq q_j \leq 1$, $0 \leq v_j \leq 10^6$), meaning that F_i has probability of q_j to have value v_j. The values v_j may be given as decimal fractions or in scientific notation. It is guaranteed that $\sum_{j=1}^{k} q_j = 1$.

The total sum of the values of k on these n lines will not exceed $3 \cdot 10^5$. The total size of the input will not exceed 5 mebibytes.

Output

Output a single real number: the minimal expected revenue among all possible joint distributions. Your answer will be considered correct if and only if its absolute or relative error does not exceed 10^{-6}.

Examples

standard input	standard output
2 2 5 4 0.254 5 0.227 8 0.269 10 0.25 9 4 0.274 9 0.272 9 0.223 8 0.231 7	2.0000000000
2 7 7 2 0.5 1 0.5 0 2 0.3 5 0.7 1	0.0000000000
1 5 1 1.0 5	5.0000000000

Problem 5. Rectangle Painting

Input file:	`standard input`
Output file:	`standard output`
Time limit:	12 seconds
Memory limit:	1024 megabytes

There is an cell grid infinite in left, right, and upwards directions (all the cells with coordinates (x, y) with $x \in \mathbb{Z}$, $y \geq 0$ exist). Initially all the cells are white. You have to process q queries of two types:

1. y_i l_i r_i: paint all cells (x, y_i) for $l_i \leq x \leq r_i$ black. If the cell is already black, its color doesn't change.
2. l_i r_i: consider all cells with x coordinate on the segment $[l_i; r_i]$. Find the highest cell such that all cells exactly under it are black. Formally, you have to find maximal h such that $\exists\, x \in [l_i; r_i]\, \forall\, y \in [0; h)$ cell (x, y) is black.

To enforce processing the queries online they are encrypted using previous answers.

Input

The first line contains one integer q ($1 \leq q \leq 10^5$) — the number of queries to process.

The next q lines will contain encrypted descriptions of queries. Let S be the sum of answers to all queries of second type processed so far.

Each description has form either 1 $(y_i \oplus S)$ $(l_i \oplus S)$ $(r_i \oplus S)$ or 2 $(l_i \oplus S)$ $(r_i \oplus S)$. It is guaranteed that $0 \leq y_i \leq 2 \cdot 10^5$, $0 \leq l_i \leq r_i \leq 2 \cdot 10^5$. Note that the guarantees are given on parameters after decryption, the numbers in input might not fit in 32-bit integers.

Don't forget to add the new answer to S after each query of the second type.

Output

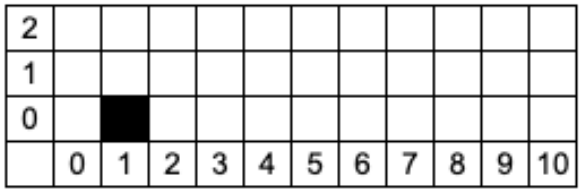
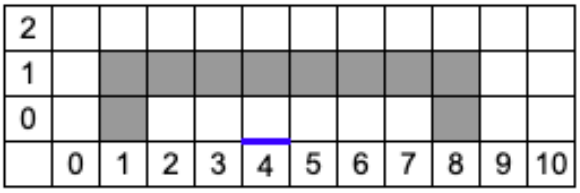
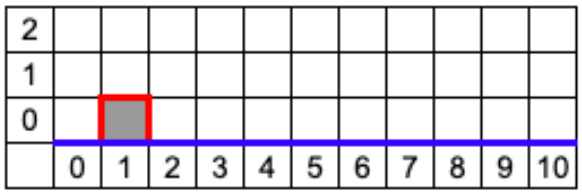
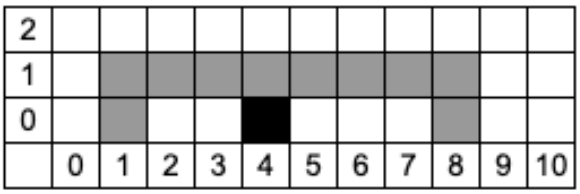
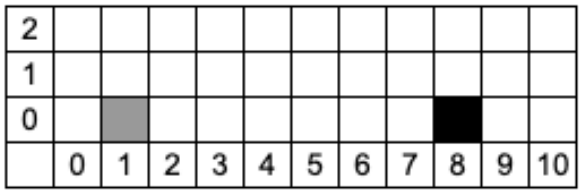
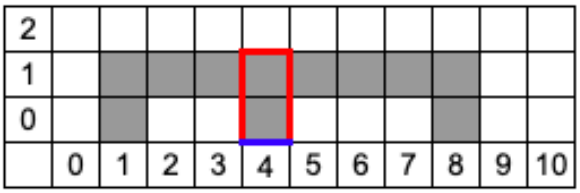
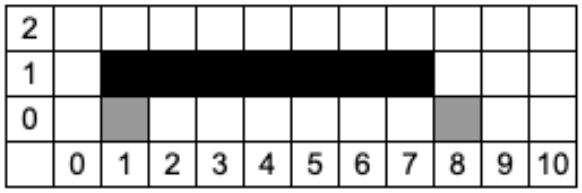
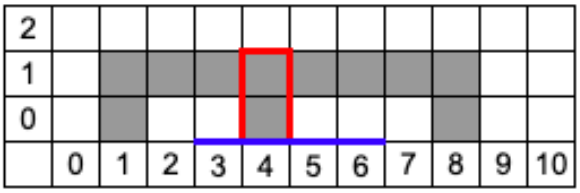
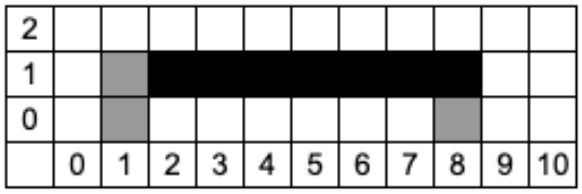
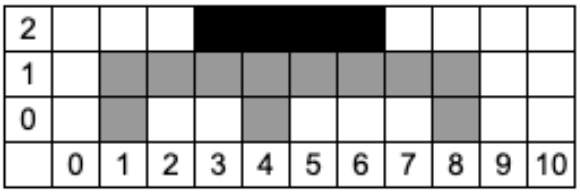
Print the answers to all queries of the second type on separate lines.

Example

standard input	standard output
10 1 0 1 1 2 0 10 1 1 9 9 1 0 0 6 1 0 3 9 2 5 5 1 1 5 5 2 5 5 2 0 5 1 7 6 3	1 0 2 2

Note

S	Encrypted	Query	Ans
0	1 0 1 1	1 0 1 1	–
0	2 0 10	2 0 10	1
1	1 1 9 9	1 0 8 8	–
1	1 0 0 6	1 1 1 7	–
1	1 0 3 9	1 1 2 8	–
1	2 5 5	2 4 4	0
1	1 1 5 5	1 0 4 4	–
1	2 5 5	2 4 4	2
3	2 0 5	2 3 6	2
5	1 7 6 3	1 2 3 6	–

Problem 7. The Hash Table

Input file:	*standard input*
Output file:	*standard output*
Time limit:	1 second
Memory limit:	512 mebibytes

There is a hash table with m slots, numbered from 0 to $m - 1$. Initially the slots are empty.

There are n elements, numbered from 0 to $n - 1$, which should be inserted into the hash table in this order.

The hash function is $h(x) = x^2 \bmod m$, so the element number i will be inserted into the slot numbered $(i^2 \bmod m)$.

Because of the strange implementation, inserting an element into a slot costs T, where T is the number of elements this slot already contains. Please compute the total cost of inserting all these n elements into the table.

Input

The first line contains an integer t, denoting the number of test cases ($1 \le t \le 5$).

Each test case is given on a single line with two integers, n and m ($1 \le n \le 10^9$, $2 \le m \le 10^9$).

Output

For each test case, print a single line containing the answer.

Example

standard input	standard output
3 5 4 1234 5678 5 4	4 229 4

Note

In the first test case, the elements $0, 1, 2, 3, 4$ are inserted into slots $0, 1, 0, 1, 0$ respectively, incurring costs of $0 + 0 + 1 + 1 + 2 = 4$.

Problem 11. Number of Colorful Matchings

Input file:	*standard input*
Output file:	*standard output*
Time limit:	2 seconds
Memory limit:	512 mebibytes

You are given a graph G with n black nodes and n white nodes, where every edge can only connect a black node and a white node (in other words, the graph is bipartite).

Each edge in G has a color: either blue or red. No two edges of the same color can connect the same pair of vertices (in other words, there are no same-color parallel edges).

For every k from 0 to n, please count the number of perfect matchings in G that contain exactly k red edges and $n - k$ blue edges. Recall that a perfect matching is a subset of n edges in which no two edges can share a common endpoint. Since the number could be large, you are only required to output the answers modulo 2.

Input

The first line contains a non-negative integer n ($1 \le n \le 300$).

Each of the next n lines contains n characters with no spaces. Together, these lines describe the adjacency matrix of red edges. The j-th character on the i-th line is "1" if there is one red edge connecting the i-th black node and the j-th white node, and "0" otherwise.

The next n lines describe the adjacency matrix of blue edges, in the same format as above.

Output

Output $n + 1$ lines containing your answers for $k = 0, 1, 2, \ldots, n$ respectively. Remember that you only need to output the answer modulo 2.

Example

standard input	standard output
2 11 10 00 11	0 0 1

Note

In the example, there exist three perfect matchings:

1. red $(1, 1)$, blue $(2, 2)$
2. red $(1, 2)$, blue $(2, 1)$
3. red $(1, 2)$, red $(2, 1)$

Problem 13. Business Semiconductor Units

Input file:	*standard input*
Output file:	*standard output*
Time limit:	1 second
Memory limit:	512 mebibytes

Business Semiconductor Units (BSU) is a large international corporation that focuses on selling fast and reliable computers to business clients. Recently, they have decided to develop a new processor model which will work even faster and more reliably than its predecessors.

The R&D department of the company is responsible for designing the instruction set and processor architecture. After the deadline, they should demonstrate the working prototype to the head of the company. Unfortunately, the whole department was playing *Minecraft* most of the time instead of doing their job, so the presented prototype supports only three simple instructions.

Let's take a closer look at their masterpiece. The new processor has 16 registers named from `r0` to `r15`, each of them can store an unsigned 16-bit integer. There is also main memory consisting of $2^{16} + 1$ eight-bit cells.

The *program* for this processor is a sequence of instructions. The instructions are executed sequentially, neither jumps nor loops are supported. The processor executes the same sequence of instructions 5000 times. That is, the following procedure is repeated 5000 times: go over the instructions from the start to the end and execute them.

Below you can see the list of available instructions. For clarity, let's call $x \bmod 2^8$ the *lower* part of the number x, and $\left\lfloor \frac{x}{2^8} \right\rfloor$ the *upper* part of the number x. The number in the i-th main memory cell is denoted mem_i.

- `imm r, b`: load the constant number b ($0 \le b < 2^{16}$) into the register named r;
- `ld x, y`: suppose that the register named y stores the number b. Then, the number $mem_{b+1} \cdot 2^8 + mem_b$ is put into the register x;
- `st x, y`: suppose that the register named x stores the number a, and the register y stores the number b. Then, the lower part of b is put into mem_a, and the upper part of b is put into mem_{a+1}.

As you can see, the instruction set is pretty lean, and the R&D department is unsure whether the processor is capable of doing anything non-trivial or not. To make it run some useful programs, they hired you and gave you an assignment. Now, you need to write a program for the new processor that multiplies n non-negative 16-bit numbers modulo 2^{16}.

Input

This problem has no input data.

Output

Output the required program in the following format.

The first line must contain an integer s, the number of instructions in your program ($1 \le s \le 10^5$).

Each of the following s lines must contain a processor instruction. The format of instructions is described above. Be careful and follow the format strictly. All the register names must be valid (that means, from `r0` to `r15`).

Interaction Protocol

Technically, this problem is an output-only interactive problem. [Sounds weird, isn't it? :)]

In each test, the interactor first reads the instructions you wrote to the output. Next, it reads the integer n and n integers a_i from the test ($1 \le n \le 4000$, $0 \le a_i < 2^{16}$). The number n is placed into the register

`r0`. For each $1 \le i \le n$, the lower part of a_i is placed into $mem_{2 \cdot i-2}$, and the upper part is placed into $mem_{2 \cdot i-1}$. All other registers and memory cells are zeroed initially.

Then, the interactor executes your program. The instructions are performed sequentially, and the execution of the program is performed exactly 5000 times.

After that, the interactor reads your answer from the register `r0` and compares it with $a_1 \cdot a_2 \cdot \ldots \cdot a_n$ modulo 2^{16}.

Example

standard input	standard output
	4 imm r3, 42 imm r1, 6 st r3, r1 ld r0, r3

Note

The output in the example does not multiply integers, so submitting this program will give you the "`Wrong Answer`" verdict. The program is only provided to illustrate the output format.

Problem 17. Count Min Ratio

Input file:	*standard input*
Output file:	*standard output*
Time limit:	4 seconds
Memory limit:	1024 mebibytes

You have R red balls, B blue balls, and one green ball. You are going to arrange the balls in a row. The score of an arrangement is defined as follows:

- Let $l_{\mathrm{R}}, l_{\mathrm{B}}, r_{\mathrm{R}}, r_{\mathrm{B}}$ be the number of red/blue balls to the left/right of the green ball, respectively. Then, the score is the maximum integer x such that $l_{\mathrm{B}} \times x \leq l_{\mathrm{R}}$ and $r_{\mathrm{B}} \times x \leq r_{\mathrm{R}}$.

Find the sum of scores of all possible arrangements, modulo 998244353. Note that balls of the same color cannot be distinguished, thus two arrangements are considered different if and only if there is such an i that the color of the i-th ball in the first arrangement differs from that of the second.

Input

The first line contains integers R ($1 \leq R \leq 10^{18}$) and B ($1 \leq B \leq 10^{6}$).

Output

Print the answer.

Examples

standard input	standard output
10 3	8390
3 10	0
100 10	801171977
999999999999999999 999999	448294209

Problem 19. ASCII Automata Art

Input file:	`standard input`
Output file:	`standard output`
Time limit:	3 seconds
Memory limit:	512 megabytes

This problem statement is quite wordy by itself and does not need a legend. You are given a regular expression and your task is to render its corresponding automaton as an ASCII art text drawing following the specification in the problem statement. Please, see examples.

A regular expression in this problem consists of uppercase letters from `A` to `Z`, special characters `+`, `?`, `*`, and parenthesis that are used for grouping. An input to the problem is given by an ⟨*input*⟩ non-terminal of the following BNF grammar:

⟨*input*⟩ ::= ⟨*expr*⟩

⟨*expr*⟩ ::= ⟨*term*⟩ | ⟨*term*⟩ '`|`' ⟨*expr*⟩

⟨*term*⟩ ::= ⟨*atom*⟩ | ⟨*atom*⟩⟨*term*⟩ | ⟨*term*⟩⟨*atom*⟩

⟨*atom*⟩ ::= ⟨*letter*⟩ | '`(`' ⟨*expr*⟩ '`)`' | ⟨*atom*⟩ '`+`' | ⟨*atom*⟩ '`?`' | ⟨*atom*⟩ '`*`'

⟨*letter*⟩ ::= '`A`' | '`B`' | ... | '`Z`'

A regular expression is rendered as an ASCII art picture using the precise rules that are given below. They recursively define a unique representation for each regular expression as a rectangular *box* of characters with the specified number of rows and columns. Empty characters of the representation, including trailing ones, must be filled with spaces.

A ⟨*term*⟩ that consists of a sequence of n uppercase letters is rendered as a box of 3 rows and $4 + n$ columns using `+` and `-` characters to render a border on the first and the last rows and columns as shown in the example. The production rule for the ⟨*term*⟩ non-terminal in the grammar is intentionally ambiguous. The longest possible sequence of adjacent ⟨*letter*⟩ non-terminals in the regular expression must be grouped into a ⟨*term*⟩ and rendered as a single box. For example, a ⟨*term*⟩ of '`NERC`' is rendered as the following 3×8 box:

```
+------+
+ NERC +
+------+
```

A ⟨*term*⟩ that consists of a sequence of ⟨*atom*⟩ non-terminals and ⟨*term*⟩ non-terminals with letters (as described above) is rendered by laying out the constituent boxes left-to-right, aligned vertically to the top, with 2 columns separating adjacent boxes. The height of the resulting box is equal to the maximum height of the constituent boxes. Each pair of adjacent boxes is joined by rendering `->` characters on the 2nd row in the columns between them. For example, a ⟨*term*⟩ of '`N(E)RC`' (consisting of a sequence: ⟨*atom*⟩ of '`A`', ⟨*atom*⟩ of '`(E)`', and a letters-only ⟨*term*⟩ of '`RC`') is rendered as the following 3×20 box:

```
+---+  +---+  +----+
+ N +->+ E +->+ RC +
+---+  +---+  +----+
```

An ⟨*expr*⟩ that consists of a single ⟨*term*⟩ is rendered as its ⟨*term*⟩.

An ⟨*expr*⟩ that consists of a '`|`'-separated sequence of ⟨*term*⟩ non-literals is rendered by laying out the corresponding ⟨*term*⟩ boxes top-to-bottom, aligned to the left, with a single row separating adjacent ⟨*term*⟩ boxes. The width of the resulting box is equal to the maximum width of the ⟨*term*⟩ boxes plus 6. There are 3 additional columns on the left, and 3 on the right. The first column and the last column use

`+` and `|` characters to join the 2nd rows of all the ⟨*term*⟩ boxes from the top to the bottom one, with `+` placed on the 2nd row of each ⟨*term*⟩ box. The 2nd and the 3rd columns on the left and the 3rd-to-last and the 2nd-to-last columns on the right have `->` characters on the 2nd rows of the corresponding ⟨*term*⟩ boxes. Additionally, shorter ⟨*term*⟩ boxes are connected on the right with extra `-` characters on their 2nd rows. For example, an ⟨*expr*⟩ of '`C|ON|TEST`' is rendered as the following 11×14 box:

```
   +---+
+->+ C +---->+
|  +---+     |
|            |
|  +----+    |
+->+ ON +--->+
|  +----+    |
|            |
|  +------+  |
+->+ TEST +->+
   +------+
```

An ⟨*atom*⟩ of '`(`' ⟨*expr*⟩ '`)`' is rendered as its ⟨*expr*⟩.

An ⟨*atom*⟩ of ⟨*atom*⟩ '`+`' is rendered as a box of its source ⟨*atom*⟩ with 2 additional rows at the bottom and 6 additional columns (3 on the left and 3 on the right). The first and the last columns, starting with the 2nd row, and the last row are filled with the connecting characters as shown in the example.

- The 2nd row starts with `+->` and ends with `->+` to connect to the 2nd row of the source ⟨*atom*⟩ box.
- The last row starts with `+<-` to represent a backwards edge in the automaton.

For example, an ⟨*atom*⟩ of '`A+`' is rendered as the following 5×11 box.

```
   +---+
+->+ A +->+
|  +---+  |
|         |
+<--------+
```

An ⟨*atom*⟩ of ⟨*atom*⟩ '`?`' is rendered as a box of its source ⟨*atom*⟩ with 3 additional rows at the top and 6 additional columns (3 on the left and 3 on the right). The first and the last columns (from the 2nd to the 5th row) and the 2nd row are filled with the connecting characters as shown in the example.

- The first row of ⟨*atom*⟩ '`?`' is always empty (filled with spaces).
- The 2nd row ends with `->+` to represent an epsilon-edge in the corresponding automaton.
- The 5th row starts with `+->` and ends with `->+` to connect to the 2nd row of the source ⟨*atom*⟩ box.

For example, an ⟨*atom*⟩ of '`B?`' is rendered as the following 6×11 box.

```
+-------->+
|         |
|  +---+  |
+->+ B +->+
   +---+
```

An ⟨*atom*⟩ of ⟨*atom*⟩ '`*`' is rendered as a box of its source ⟨*atom*⟩ with 5 additional rows (3 at the top and 2 at the bottom) and 6 additional columns (3 on the left and 3 on the right). The first and the last columns, with the 2nd and the last row, are filled with the connecting characters as shown in the example.

- The first row of ⟨*atom*⟩ '`*`' is always empty (filled with spaces).

- The 2nd row ends with `->+` to represent an epsilon-edge in the corresponding automaton.
- The 5th row starts with `+->` and ends with `->+` to connect to the 2nd row of the source ⟨*atom*⟩ box.
- The last row starts with `+<-` to represent a backwards edge in the automata.

For example, an ⟨*atom*⟩ of '`C*`' is rendered as the following 8×11 box.

```
+--------->+
|          |
|  +---+   |
+->+ C +->+
|  +---+   |
|          |
+<---------+
```

An ⟨*input*⟩ is rendered as a box that has 6 more columns than the corresponding box of the ⟨*expr*⟩, with 3 additional columns on the left, and 3 on the right. The 2nd row starts with `S->` to represent the starting state of the automaton and ends with `->F` to represent the final state of the automaton. See the example output.

Input

The input consists of a single line that corresponds to the ⟨*input*⟩ non-terminal of the grammar given the problem statement and has at most 100 characters in length.

Output

On the first line of the output, write two integers h and w — the height and the width, correspondingly, of the $h \times w$ box that corresponds to the given ⟨*input*⟩. On each of the next h lines, write w characters of the corresponding ASCII art rendering.

Example

standard input

```
NE?(ER)C++|(IS)*?|(CHA((LL))ENGING)
```

standard output

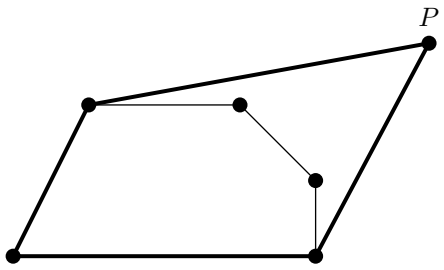
```
23 57
      +---+               +----+        +---+
S->+->+ N +->+-------->+->+ ER +->+->+->+ C +->+->+->+->F
   |  +---+  |         |  +----+  |  |  +---+  |  |  |
   |         |  +---+  |          |  |         |  |  |
   |         +->+ E +->+          |  +<--------+  |  |
   |            +---+             |               |  |
   |                              +<--------------+  |
   |                                                 |
   |                                                 |
   +->+--------------->+----------------------------->+
   |  |                |                              |
   |  |                |                              |
   |  +->+--------->+->+                              |
   |     |          |                                 |
   |     |  +----+  |                                 |
   |     +->+ IS +->+                                 |
   |     |  +----+  |                                 |
   |     |          |                                 |
   |     +<---------+                                 |
   |                                                 |
   |  +-----+  +----+  +--------+                     |
   +->+ CHA +->+ LL +->+ ENGING +-------------------->+
      +-----+  +----+  +--------+
```

Problem 23. Fiber Shape

Input file:	`standard input`
Output file:	`standard output`
Time limit:	3 seconds
Memory limit:	512 megabytes

Imagine a board with n pins put into it, the i-th pin is located at (x_i, y_i). For simplicity, we will restrict the problem to the case where the pins are placed in vertices of a convex polygon.

Then, take a non-stretchable string of length l, and put it around all the pins. Place a pencil inside the string and draw a curve around the pins, trying to pull the string in every possible direction. The picture below shows an example of a string tied around the pins and pulled by a pencil (a point P).



Your task is to find an area inside this curve. Formally, for a given convex polygon S and a length l let's define a *fiber shape* $F(S, l)$ as a set of points t such that the perimeter of the convex hull of $S \cup \{t\}$ does not exceed l. Find an area of $F(S, l)$.

Input

The first line contains two integers n and l ($3 \le n \le 10^4$; $1 \le l \le 8 \cdot 10^5$) — the number of vertices of the polygon S and the length of the string. Next n lines contain integers x_i and y_i ($-10^5 \le x_i, y_i \le 10^5$) — coordinates of polygon's vertices in counterclockwise order. All internal angles of the polygon are strictly less than π. The length l exceeds the perimeter of the polygon by at least 10^{-3}.

Output

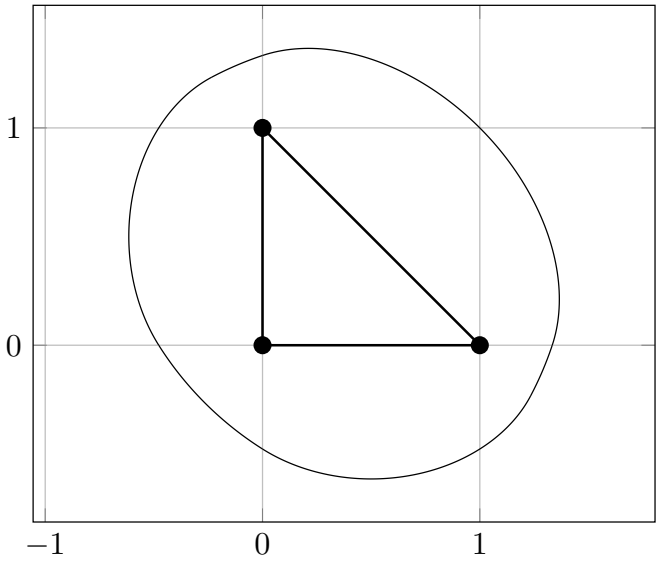
Output a single floating-point number — the area of the fiber shape $F(S, l)$. Your answer will be considered correct if its absolute or relative error doesn't exceed 10^{-6}.

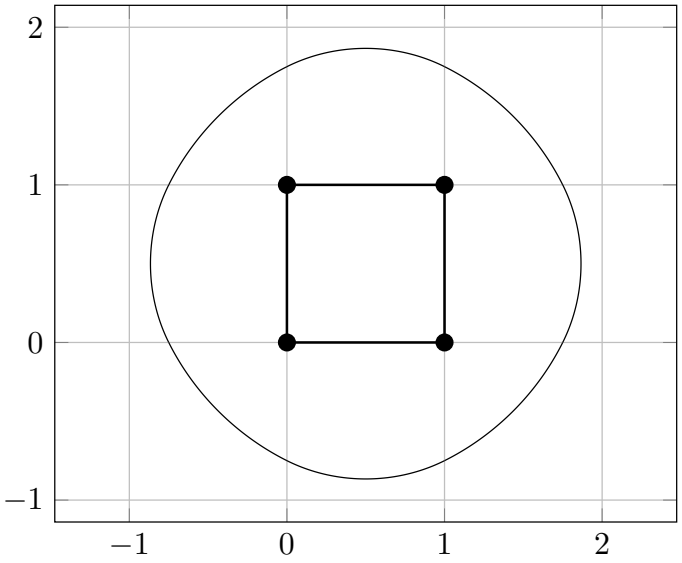
Examples

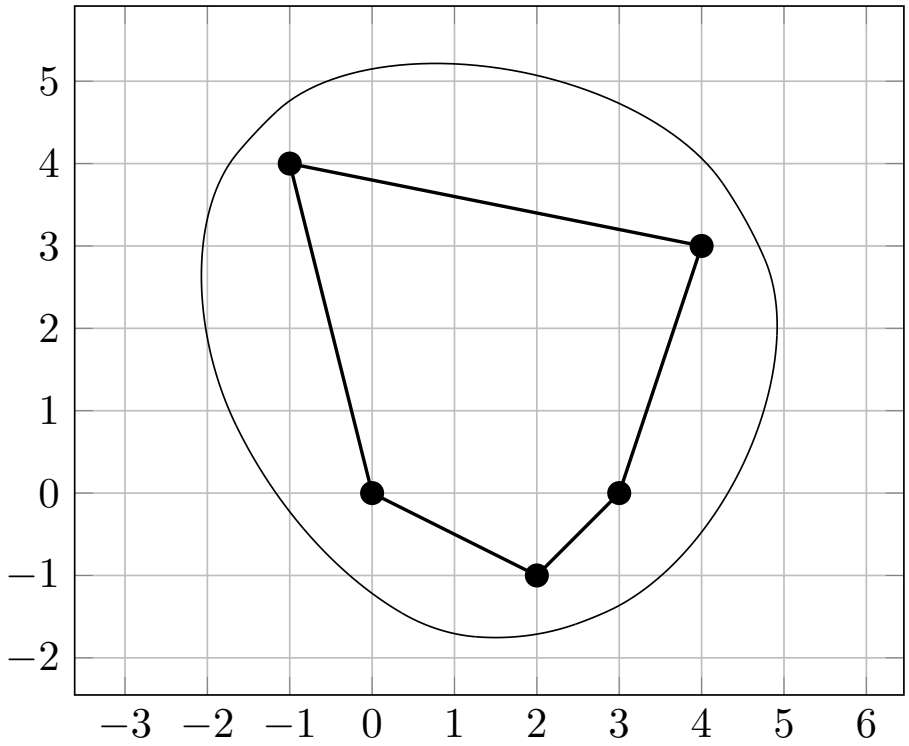
standard input	standard output
3 4 0 0 1 0 0 1	3.012712585980357
4 5 0 0 1 0 1 1 0 1	5.682061989789656
5 17 0 0 2 -1 3 0 4 3 -1 4	37.719371276930820

Note

The following pictures illustrate the example tests.







Problem 29. Hard Optimization

Input file:	`standard input`
Output file:	`standard output`
Time limit:	3 seconds
Memory limit:	512 megabytes

You are given a set of n segments on a line $[L_i; R_i]$. All $2n$ segment endpoints are pairwise distinct integers.

The set is *laminar* — any two segments are either disjoint or one of them contains the other.

Choose a non-empty subsegment $[l_i, r_i]$ with integer endpoints in each segment ($L_i \le l_i < r_i \le R_i$) in such a way that no two subsegments intersect (they are allowed to have common endpoints though) and the sum of their lengths ($\sum_{i=1}^{n} r_i - l_i$) is maximized.

Input

The first line contains a single integer n ($1 \le n \le 2 \cdot 10^3$) — the number of segments.

The i-th of the next n lines contains two integers L_i and R_i ($0 \le L_i < R_i \le 10^9$) — the endpoints of the i-th segment.

All the given $2n$ segment endpoints are distinct. The set of segments is laminar.

Output

On the first line, output the maximum possible sum of subsegment lengths.

On the i-th of the next n lines, output two integers l_i and r_i ($L_i \le l_i < r_i \le R_i$), denoting the chosen subsegment of the i-th segment.

Example

standard input	standard output
4 1 10 2 3 5 9 6 7	7 3 6 2 3 7 9 6 7

Note

The example input and the example output are illustrated below.

Problem 31. Newton's Symbol

Input file:	*standard input*
Output file:	*standard output*
Time limit:	4 seconds
Memory limit:	512 mebibytes

Newton's symbol, also known as binomial coefficient, which is written as $\binom{n}{k}$, denotes the number of ways to choose k objects out of n distinguishable objects. If $k < 0$ or $k > n$, then $\binom{n}{k} = 0$, otherwise

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

You are given two n-element sequences $a_1, \ldots, a_n$ and $b_1, \ldots, b_n$. Your task is to answer q queries: i-th query consists of two integers c_i and d_i and answer to it is

$$\sum_{j=1}^{n} \binom{c_i - a_j}{d_i - b_j}.$$

Note: After peeking to some other problems we noted that it is an often practice to ask contestants to print the result modulo some prime number when the actual result could be really big. We thought we will steal this idea and decided to ask you to print the result **modulo prime number 2.**

Additionally, queries are encrypted and in order to get to know c_i and d_i you need to know remainders modulo 2 of results of all previous queries.

Input

In the first line of the input there is one integer n ($1 \le n \le 300\,000$). The following n lines describe sequences a and b: in i-th of them there are two integers a_i and b_i ($0 \le a_i, b_i \le 300\,000$).

The next line contains a single integer q ($1 \le q \le 300\,000$). The following q lines describe queries: in i-th of them there are two integers c'_i and d'_i ($0 \le c'_i, d'_i \le 300\,000$).

In order to get to know actual values of c_i and d_i you need to initialize two auxiliary variables X and Y, initially set to 0. After processing $i - 1$ queries you can decrypt i-th query according to the following formulas:

$$c_i = (c'_i + X) \bmod 300\,001, \qquad d_i = (d'_i + Y) \bmod 300\,001.$$

If the answer to i-th query is even then set $X := X + c_i$; otherwise set $Y := Y + d_i$.

Note: the values of X and Y may not fit into 32-bit variables.

Output

You should print exactly q lines in the output: i-th of them should contain one integer – **the remainder modulo 2 of the answer to *i*-th query.**

Example

standard input	standard output
4 0 0 1 3 3 2 4 4 6 0 0 3 4 299998 3 0 0 7 10 2 15	1 0 0 1 0 1

Explanation to the sample: The queries after decryption written as (c_i, d_i) are following: $(0, 0)$, $(3, 4)$, $(0, 3)$, $(3, 0)$, $(10, 10)$ and $(15, 15)$ and full answers to them are 1, 2, 0, 1, 38 and 93.

Problem 37. Gigantic tree

Input file:	*standard input*
Output file:	*standard output*
Time limit:	9 seconds
Memory limit:	1024 mebibytes

Byteasar bought a gigantic tree for her girlfriend Algoline. Surely an unusual gift, however Byteasar is an algorithmician and Algoline got used to such surprises.

As expected, this tree was not a plant, but a connected graph without cycles. It is huge, but it can be described in a concise way. Its vertices are arranged into n layers. The first layer contains only one vertex – its root. Each vertex has children in the next layer with exception of vertices in the last layer which are leaves. For each i from interval $[1, n-1]$ each vertex from i-th layer has a_i children.

Algoline wants to show Byteasar how satisfied she is with this gift, so she decided to play a game with him. She chose a vertex A and Byteasar chose a vertex B (possibly the same one). Now they are going to take turns alternately. In each turn they will color one vertex. Algoline uses red paint and Byteasar uses blue paint. On the beginning of the game all vertices are white. Each vertex (including A and B) will be colored exactly once.

Once all vertices are colored either red or blue, Algoline and Byteasar will count their scores. The result of Algoline (denoted by S_A) is the sum of distances of all red vertices from A and the result of Byteasar (denoted by S_B) is the sum of distances of all blue vertices from B. Distance between two vertices is measured as the number of edges on the shortest path between them. Algoline's goal is to maximize the difference $S_A - S_B$ and Byteasar's goal is to minimize it.

Byteasar quickly realized that this is a finite game with full information and it is possible to determine this difference assuming that they will both play optimally. He wants you to help him and calculate this difference. As it can be very large, it suffices to determine its remainder modulo $10^9 + 7$.

Additionally, as it will be mean to forget about the gift after just one game, you should determine this difference for many options of choosing a starting pair of vertices A and B.

Input

In the first line of the input there are two integers n and q ($2 \le n \le 300\,000$, $1 \le q \le 300\,000$) denoting the number of layers in the tree and the number of pairs of starting vertices that you need to consider.

In the second line of the input there is a sequence of $n-1$ integers $a_1, a_2, \ldots, a_{n-1}$ ($2 \le a_i \le 300\,000$) as described in the statement.

In following q lines there are descriptions of vertices A and B chosen in following games. It is possible to prove that the final difference depends only on the layer where A is located, on the layer where B is located and the layer where the lowest common ancestor of A and B is located. Because of that, you will be given only the indices of these layers denoted by W_A, W_B and $W_{\mathrm{LCA}(A,B)}$ ($1 \le W_{\mathrm{LCA}(A,B)} \le W_A, W_B \le n$).

Output

You should print q lines in the output, where i-th line should contain the final difference in i-th game modulo $10^9 + 7$.

Example

standard input	standard output
3 3 3 2 3 2 1 1 1 1 2 3 2	4 1 1000000003

Explanation to the sample: The tree from the sample test contains three layers and contains one vertex in the first layer, three vertices in the second layer and six vertices in the third layer.

In the second game both Algoline and Byteasar chose the root as their vertex. In the optimally played game they should choose vertices in the order of nonascending indices of their layers what will result in the difference being $(2 + 2 + 2 + 1 + 1) - (2 + 2 + 2 + 1 + 0) = 1$.

The third game will end up with result -4. Note that in such case you should print $-4 \bmod (10^9 + 7) = 10^9 + 3$.

Problem 41. Kingdoms and Quarantine

Input file:	*standard input*
Output file:	*standard output*
Time limit:	8 seconds
Memory limit:	512 mebibytes

There are two kingdoms A (with N_1 cities) and B (with N_2 cities), and M bidirectional roads, each connecting a city from A and a city from B, such that there is no more than one road connecting any pair of cities.

The cities in the kingdom A are enumerated from 1 to N_1, and the cities in the kingdom B are enumerated from $N_1 + 1$ to $N_1 + N_2$. The roads are enumerated from 1 to M; the road i connects two cities a_i and b_i, where a_i and b_i satisfy $1 \le a_i \le N_1$ and $N_1 + 1 \le b_i \le N_1 + N_2$.

Once upon a time, a dangerous virus appeared in one kingdom, so the Kings decided to close some roads.

Let D_j be the initial number of roads connecting the city j with other cities, and d_j be the number of currently active (not closed) roads connecting the city j with other cities.

The road x can be closed if and only if following conditions are met **before** closing the road:

- It was not closed before.
- The numbers d_{a_x} and D_{b_x} must have the same parity (both even or both odd).
- The numbers d_{b_x} and D_{a_x} must have the same parity (both even or both odd).

Find the maximum number of roads that can be closed, and then find a sequence of road closing operations such that this maximum is achieved.

Input

The first line of input contains three integers, N_1, N_2, and M: the number of cities in kingdom A, the number of cities in kingdom B, and the number of roads, respectively ($1 \le N_1, N_2, M \le 3000$, $1 \le M \le N_1 \cdot N_2$).

The i-th of the following M lines describes the road i and contains two integers a_i and b_i ($1 \le a_i \le N_1$, $N_1 + 1 \le b_i \le N_1 + N_2$): the numbers of cities connected by that road. You may assume that, for different i and j, $a_i \ne a_j$ or $b_i \ne b_j$.

Output

On the first line, print the integer K: the maximum number of roads that can be closed. On the second line, print K integers r_i ($1 \le r_i \le M$): the numbers of roads to be closed, in the order of closing them.

If there are several optimal answers, print any one of them.

Examples

standard input	standard output
2 3 5 1 3 1 4 1 5 2 4 2 5	3 1 4 2
1 2 2 1 2 1 3	0
4 3 7 1 5 2 5 2 6 2 7 3 6 4 5 4 7	5 1 7 6 2 4

Note

In the first example, $D_1 = 3$, $D_2 = 2$, $D_3 = 1$, $D_4 = 2$, $D_5 = 2$.

Initially, $d_1 = 3$, $d_2 = 2$, $d_3 = 1$, $d_4 = 2$, $d_5 = 2$, so we can close the following roads:

- Road 1 connecting city 1 and city 3.
- Road 4 connecting city 2 and city 4.
- Road 5 connecting city 2 and city 5.

Let us close road 1, then

$d_1 = 2$, $d_2 = 2$, $d_3 = 0$, $d_4 = 2$, $d_5 = 2$.

After that, the roads that can be closed are the following:

- Road 4 connecting city 2 and city 4.
- Road 5 connecting city 2 and city 5.

Let us close road 4, then

$d_1 = 2$, $d_2 = 1$, $d_3 = 0$, $d_4 = 1$, $d_5 = 2$.

Now, we can close only road 2, connecting city 1 and city 4.

After that, $d_1 = 1$, $d_2 = 1$, $d_3 = 0$, $d_4 = 0$, $d_5 = 2$.

It can be shown that it is impossible to close more than three roads, so the answer is 3.

Problem 43. Lazy Judge

Input file:	*standard input*
Output file:	*standard output*
Time limit:	4 seconds
Memory limit:	512 mebibytes

This is an interactive problem.

The judges are working on the strategy for the jury program for the modified version of the problem **J** from the current contest.

In that problem, Alice secretly invents a permutation of first N integers $a_1, a_2, \ldots a_N$ and tells N to Bob.

Bob asks some questions to identify this permutation. Alice may change the permutation in the process as long as it is consistent with her previous answers.

The judges are planning to create an AliceBot that will do the following.

There are two phases: the question phase and the answer phase.

In the question phase, the judge tells to AliceBot an integer N. Then AliceBot has to answer some questions the judge asks about the permutation.

In the subsequent answer phase, AliceBot must compose two different permutations $a_1, \ldots, a_N$ and $b_1, \ldots, b_N$ that are consistent with the answers from the previous phase.

The judge who asks questions has an initial patience $P = 2N$. Each time the judge asks a question, the judge's patience decreases.

There are three types of questions the judge can ask:

- Type 1, formatted as "`? 1` i j k": the judge chooses three different integers i, j, k ($1 \le i, j, k \le N$), AliceBot looks at the three integers a_i, a_j, and a_k, and tells Bob the **value** of their median (the one that is neither minimum nor maximum). Each such question decreases the judge's patience by 2.
- Type 2, formatted as "`? 2` i j": the judge chooses two different integers i, j ($1 \le i, j \le N$), and AliceBot answers i if $a_i < a_j$, or j otherwise. Each such question decreases the judge's patience by 2.
- Type 3, formatted as "`? 3` i j": the judge chooses two different integers i, j ($1 \le i, j \le N$), and AliceBot tells the minimum **value** among a_i and a_j. Each such question decreases the judge's patience by 1.

You may assume that judge's patience will be **strictly greater** than 2 after asking a question. When the judge decides that they asked enough questions, the command "`!`" is sent to the AliceBot, switching it to the answer phase.

In the answer phase, AliceBot tells the judge two permutations $a_1, \ldots, a_N$ and $b_1, \ldots, b_N$. These two permutations must be consistent with all the answers given in the question phase, and the number of positions i such that $a_i \neq b_i$ has to be at least $\lceil p/2 \rceil$, where p is the judge's patience at the end of the question phase.

Because the judge is too lazy, you are asked to implement the AliceBot. It can be shown that the problem is solvable for every possible N from the constraints.

Interaction Protocol

First, the jury program tells you one integer N on a separate line ($4 \le N \le 50\,000$).

Then the jury asks questions.

A question of type 1 is a line formatted as "`? 1` i j k" ($1 \le i, j, k \le N$; i, j, and k are pairwise distinct). You then print a line with a single integer: the median of the values a_i, a_j, and a_k.

A question of type 2 is a line formatted as "`? 2` i j" ($1 \le i, j \le N$; $i \ne j$). You then print a line with a single integer: i if $a_i < a_j$, or j if $a_i > a_j$.

A question of type 3 is a line formatted as "`? 3` i j" ($1 \le i, j \le N$; $i \ne j$). You then print a line with a single integer: the minimum value among a_i and a_j.

Let there will be a total of q_1 questions of type 1, q_2 questions of type 2, and q_3 questions of type 3. You may assume that the value $p = 2N - 2q_1 - 2q_2 - q_3$ is **strictly greater** than 2.

To switch to the answering phase, the jury program issues a line consisting of the '`!`' sign. After that, you must print two lines, the first line containing N space-separated integers $a_1, \ldots, a_N$, and the second line containing N space-separated integers $b_2, \ldots, b_N$. Each of these two sequences must be a permutation of $1, \ldots, N$, and they must differ in at least $\lceil p/2 \rceil$ positions.

Don't forget to print the newline character and flush the output buffer after printing the answers and after printing each permutation, otherwise, you may get the "`Idleness Limit Exceeded`" error.

Example

standard input	standard output
5 ? 1 1 2 3 ? 2 2 4 ? 3 4 5 !	 4 4 1 3 5 4 1 2 5 4 3 2 1

Problem 47. Nimber Sequence

Input file:	*standard input*
Output file:	*standard output*
Time limit:	1 second
Memory limit:	256 mebibytes

Given are Nimbers $a_1, a_2, \ldots, a_{K-1}$, $b_1, b_2, \ldots, b_5$, and $c_1, c_2, \ldots, c_5$.

Let $a_n = \left(\bigoplus_{i=1}^{5} a_{n-i} \otimes b_i\right) \oplus \left(\bigoplus_{i=1}^{5} a_{n-K+i} \otimes c_i\right)$ for all $n \geq K$.

Find the value of a_m.

Note

Nimbers are non-negative integers associated with Nim games. For the purposes of this problem, two operations are necessary:

- "$\oplus$" is the Nim sum: $a \oplus b = \operatorname{mex}\big(\{a' \oplus b \mid 0 \leq a' < a\} \cup \{a \oplus b' \mid 0 \leq b' < b\}\big)$,
- "$\otimes$" is the Nim product: $a \otimes b = \operatorname{mex}\big(\{(a' \otimes b) \oplus (a \otimes b') \oplus (a' \otimes b') \mid 0 \leq a' < a,\ 0 \leq b' < b\}\big)$.

Here, $\operatorname{mex}(S)$ represents the smallest non-negative integer $d \notin S$.

Input

The first line of input contains two positive integers K and m ($6 \leq K \leq 10^5$, $1 \leq m \leq 10^{18}$).

The second line contains $K-1$ non-negative integers $a_1, a_2, \ldots, a_{K-1}$ ($0 \leq a_i < 2^{32}$).

The third line contains five non-negative integers $b_1, b_2, \ldots, b_5$ ($0 \leq b_i < 2^{32}$).

The fourth line contains five non-negative integers $c_1, c_2, \ldots, c_5$ ($0 \leq c_i < 2^{32}$).

Output

Output a single line with a single integer: the value of a_m.

Examples

standard input	standard output
6 1000000000000000000 1 2 3 4 5 1 0 0 0 0 0 0 0 0 0	5
6 10 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15	9
11 123 849 674 223 677 243 657 979 583 643 845 979 282 313 567 433 122 443 132 554 132	32098

Problem 53. Historic Breakthrough

Input file:	*standard input*
Output file:	*standard output*
Time limit:	2 seconds
Memory limit:	512 mebibytes

Rikka is taking the undergraduate cryptography course this year. In the class, it is mentioned that a new method of factorization was recently discovered, which is considered a historic breakthrough in cryptography. Rikka believes that there will be comparable algorithms for some specific class of numbers with simpler goals.

Now you are asked to design and implement such a "factorization" algorithm: given an integer m, print an integer n such that

$$m = \frac{n\varphi(n)}{2} = \sum_{\substack{1 \le i \le n, \\ \gcd(i,n)=1}} i.$$

It is guaranteed that such an n exists in all the test cases.

Input

The first line of input contains an integer T ($1 \le T \le 51$), the number of test cases.

The i-th of the following T lines contains a single integer m_i ($0 < m_i < 10^{36}$).

Output

For each test case, print an integer n_i for the corresponding m_i. It is guaranteed that the answer exists in every test case. If there are several possible answers, print any one of them.

Example

standard input	standard output
3 20 21 475750381222420656586462245096576000	10 7 1497700821900508526

Problem 59. Goldberg Machine 2

Input file:	*standard input*
Output file:	*standard output*
Time limit:	6 seconds
Memory limit:	512 mebibytes

Rube is back in business with a new mind-blowing contraption! The contraption consists of a grid with n rows and m columns, with some cells of the grid containing an arrow pointing right or down, and other cells being empty. One can place a token in the top left corner cell of the grid, and then the magic happens: the token starts moving through the cells according to the arrow directions. Further, each arrow the token passes through switches direction when the token leaves its cell. Formally, if a token is in a cell with an arrow pointing right (resp. down), it then moves to the cell immediately to the right (resp. down), and the arrow in the original cell switches to pointing down (resp. right). The movement stops once the token arrives at an empty cell, or leaves the grid altogether, at which point the token is discarded.

As an example, consider a 3×3 grid shown below on the left ("`>`" and "`v`" describe arrows pointing right and down respectively, and "`.`" describes an empty cell). Two more grids show grid states after placing a token in the top left corner of the grid, and after placing another token after that.

```
>v>    v>>    >>>
vv> -> v>> -> >>>
v.>    v.>    >.>
```

Rube has manufactured many copies of his machine and sells them for a good price. However, he has been confronted by two very sophisticated customers who argue that their machines look the same, which offends their exquisite taste for truly unique things. Indeed, their machines have the same *shape*, that is, the sets of cells containing the arrows are equal for both machines, although the arrow configurations themselves may be different.

Rube agreed to modify the machines so that, despite looking similar, machines will never end up with the same arrow configurations after feeding any number of tokens in any, or both, of them. He is now bombarded with requests from the customers on how exactly machines should be modified: each request is to change a single arrow state in one of the machines. After each request Rube needs to determine if machines can eventually arrive to the same arrow configuration, and if so, what is the smallest total number of times you need to place a token in either of the machines before that happens, distributing the tokens arbitrarily between the two machines. Note that the changes **persist**, that is, no modification is undone after giving an answer to the request.

Once again, Rube is lamenting the complexity of his inventions since the task looks incredibly difficult. Help him answer all of customers' requests and avoid being sued out of existence.

Input

The first line contains three integers n, m, q — dimensions of the grid, and the number of requests ($1 \le n, m \le 100$, $1 \le q \le 10^5$).

The following n lines describe the state of the first machine. Each of these lines contains m characters. Each of the characters is one of "`>`", "`v`", or "`.`", which describe an arrow pointing right or down, or an empty cell respectively.

The following n lines after that describe the state of the second machine in the same format. It is guaranteed that the two grids have the same shape, that is, they contains arrows (regardless of directions) in the same set of cells. Further, it is guaranteed that the top left corner of each grid contains an arrow, and that a token can potentially reach each arrow cell after placing some number of tokens in each of the machines.

The following q lines describe requests in order they are received. Each request is described by three

integers t, r, c — the number of the machine to be operated, and the row and the column indices of the cell to be switched ($t \in \{1, 2\}$, $1 \le r \le n$, $1 \le c \le m$). Rows are numbered top to bottom, and columns are numbered left to right, both starting from 1. It is guaranteed that the cell to be switched in each request contains an arrow.

Extra empty lines may be present between sections for visual clarity.

Output

Print $q + 1$ lines, each containing answers after processing the first $0, 1, 2, \ldots, q$ requests respectively, in order. If the machines in their current states can not arrive to the same arrow configuration after any number of tokens placed in either of them, print -1 as the answer. Otherwise, print the smallest total number of tokens that need to be placed in the machines before they arrive to the same configuration.

Do not print leading zeros.

Example

standard input	standard output
3 3 9 >v> vv> v.> v>> v>> v.> 2 3 1 2 2 1 2 1 1 1 3 3 1 2 2 2 3 3 2 3 1 1 1 2 1 2 1	1 -1 -1 2 14 -1 -1 -1 -1 0

Problem 61. Crab's Cannon

Input file:	*standard input*
Output file:	*standard output*
Time limit:	3 seconds
Memory limit:	512 mebibytes

Mr. Crab is a famous engineer. He has recently invented a new, very powerful weapon called *The Palindromic Cannon*. To make a shot from this mighty cannon, one has to load it with some string consisting of ℓ letters from **unlimited** alphabet.

Let us denote the *palindromic prefix set (PPS)* of a string as the set of all the numbers i ($1 \le i \le \ell$) such that the prefix of length i is a palindrome. For example, the PPS for the string "`abacaba`" is $\{1, 3, 7\}$, and the PPS for the string "`aaaa`" is $\{1, 2, 3, 4\}$. The size of the PPS for the string s is called the *force* of s.

Mr. Crab wanted to test his new weapon. So, he wrote down some string s, ready to be loaded into the cannon. Then he suddenly felt very tired and fell asleep. When he woke up, he couldn't find his string, because enemy spies, Mr. Crabbarc and Mr. Barc, broke into his house and performed some operations with the string.

First, Mr. Crabbarc arrived. He took the string and wrote down its PPS in random order in a sequence. Then, Mr. Barc broke into Mr. Crab's house and might have erased some of the numbers that Mr. Crabbarc wrote.

Mr. Crab needs your help. You need to restore his string s and tell him the force of this string. Since there are many possible ways to restore the string, you need to find the minimum possible force of the restored string.

Input

Input contains muliple test cases. The format of each test case is described below.

The first line of each test case contains two integers n and ℓ ($1 \le n \le 3 \cdot 10^5$, $1 \le \ell \le 10^{18}$), the length of the sequence Mr. Crab found after Mr. Crabbarc and Mr. Barc visited his house, and the length of the initial string.

The second line of the test case contains n integers a_i ($1 \le a_i \le \ell$), the numbers in the sequence. It is guaranteed that all of them are distinct.

After the last test case, there is a line containing "`0 0`". Your program must terminate gracefully after reading this line.

It is guaranteed that the sum of n over all the test cases does not exceed $3 \cdot 10^5$.

Output

For each test case, print line with a single integer: the minimum possible force of the restored string.

Example

standard input	standard output
3 7 1 3 7 4 12 7 1 3 9 3 16 16 1 8 0 0	3 5 3

Note

In the first test case, one of the strings with the minimum possible force is "*abacaba*". Its PPS is $\{1, 3, 7\}$, so its force is 3.

In the second test case, the string could have been "*cbcbcbcbcrab*". Its PPS is $\{1, 3, 5, 7, 9\}$, so its force is 5. One can prove that it is not possible to restore a string with smaller PPS.

In the third test case, consider the string "*crabbarccrabbarc*". Its PPS is $\{1, 8, 16\}$, so its force is 3.

Problem 67. The Struggle

Input file:	*standard input*
Output file:	*standard output*
Time limit:	4.5 seconds
Memory limit:	512 mebibytes

Nocriz is a student who, like many, has a dream for his life. Unfortunately, life isn't always easy, and there are times when the dream seems far away and the pursuit feels difficult.

Reality often drives people away from their pursuit of dreams to settle for what they have at hand. Lured to set aside the struggle, Nocriz said to Artemisia, "But I shouldn't give up the dream, right?". Artemisia replied: "Of course one should not give up the dream easily! Struggle, struggle until you are crushed by a rock!".

That night, Nocriz had a very bad dream. In his dream, there was a rock in the shape of an ellipse, and he was asked to calculate the sum of values $(x \oplus y)^{33} x^{-2} y^{-1}$, performing the calculations modulo $10^9 + 7$, for all integer points (x, y) in the ellipse, where $\oplus$ is the bitwise XOR operation.

Formally, the ellipse is specified by six integers a, b, c, d, e, f. Consider the ellipse $E = \{(x, y) \mid x, y \in \mathbb{Z},\ a(x-b)^2 + c(y-d)^2 + e(x-b)(y-d) \le f\}$. It is guaranteed that all points of the ellipse satisfy $0 < x, y < 4 \cdot 10^6$, and that the ellipse contains at least one integer point. Now, consider the coordinates as residues modulo $10^9 + 7$, and calculate

$$\sum_{(x,y) \in E} (x \oplus y)^{33} x^{-2} y^{-1}.$$

In particular, considering numbers as residues means that z^{-1} is the modular inverse of z, and the result of every addition and multiplication has to be taken modulo $10^9 + 7$.

Nocriz was not crushed by the rock that day. Can you solve this problem, like Nocriz did?

Input

The first line contains an integer T ($1 \le T \le 10^4$), the number of test cases. Then T test cases follow.

The first and only line of each test case contains six integers a, b, c, d, e, f ($1 \le a, c \le 100$, $-100 \le e \le 100$, $1 \le b, d \le 4 \cdot 10^6$, $0 \le f \le 10^{15}$). It is guaranteed that all points (x, y) of the ellipse satisfy $0 < x, y < 4 \cdot 10^6$, and that the ellipse contains at least one integer point.

It is also guaranteed that the sum of the values $\max\limits_{(x,y) \in E} \max(x, y)$ over all test cases is at most $4 \cdot 10^6$.

Output

For each test case, output a single line with the integer representing the answer. Don't forget to treat coordinates as residues modulo $10^9 + 7$ in calculations.

Example

standard input	standard output
2 2 2 1 3 -1 6 13 19 11 17 6 1919	566081223 453578240

Problem 71. 3D-chans are not needed

Input file:	`standard input`
Output file:	`standard output`
Time limit:	8 seconds
Memory limit:	256 megabytes

Consider a 4-dimensional lattice cube $[1, n]^4$ and all lattice points within it, i. e., for which the coordinates range between (and including) 1 and n. (Namely, points (x_1, x_2, x_3, x_4), such that $1 \le x_i \le n$ and $x_i \in \mathbb{Z}$ for all $i = 1, 2, 3, 4$.) Also, you are given m constraints. The i-th of them is the following: the Manhattan distance to the lattice point $(y_1^i, y_2^i, y_3^i, y_4^i)$ in the cube is not greater then r_i.

Calculate the number of lattice points in the cube, which satisfy all given constraints. Since the problem doesn't look bad without this, calculate the answer modulo 998244353.

The Manhattan distance between two points $\mathbf{a} = (a_1, a_2, a_3, a_4)$ and $\mathbf{b} = (b_1, b_2, b_3, b_4)$ is defined as

$$d(\mathbf{a}, \mathbf{b}) = |a_1 - b_1| + |a_2 - b_2| + |a_3 - b_3| + |a_4 - b_4|.$$

Input

The first line contains two integers m, n ($1 \le m, n \le 2 \cdot 10^5$), the number of constraints, and the upper bound for coordinates.

The i-th of the next m lines contains 5 integers y_1^i, y_2^i, y_3^i, y_4^i and r_i ($1 \le y_j^i \le n$, $0 \le r_i \le 4n$), the description of the i-th constraint.

Since the jury is evil, you are not guaranteed anything. There may be equal points, there may be constraints that are satisfied by points outside of the cube, the answer may be zero, the answer may be nonzero. Good luck!

Output

Output the number of points modulo 998244353.

Examples

standard input	standard output
1 2 1 1 2 1 1	5
2 9 1 4 8 8 8 1 3 3 7 7	366
4 200000 89720 90321 165797 98663 492130 160473 118956 106147 16203 280661 84981 6679 185061 19011 527856 125272 154317 149362 149529 204616	231512004

Problem 73. Imprecise Permutation Sort

Input file:	`standard input`
Output file:	`standard output`
Time limit:	10 seconds
Memory limit:	512 megabytes

This is an interactive problem.

A permutation $a[1], a[2], \ldots, a[n]$ of integers from 1 to n is hidden from you.

Your task is to sort it in ascending order by comparing and swapping pairs of elements. This problem could be pretty easy, but the jury member responsible for the problem was too concentrated on floating-point arithmetic in problems G and J and implemented an "imprecise" comparator:

- if $\frac{|a[i]-a[j]|}{max(a[i],a[j])} \leq 0.01$, then return 0;
- otherwise, if $a[i] < a[j]$, then return -1;
- otherwise, return 1.

Your program can make queries to compare any two elements with this comparator, or to swap any two elements. After each swap, it will be told whether the permutation became sorted.

Sort a permutation of size up to 16 384 using no more than 300 000 queries.

Interaction Protocol

Receive an integer n from the jury's program — the size of the permutation ($1 \leq n \leq 16\,384$). Then print queries and receive responses from the jury's program. After each query the output should be flushed and then a single integer should be read — the response to that query.

Comparison queries have a format "`C i j`", and swap queries have a format "`S i j`", where i and j are indices of two elements ($1 \leq i, j \leq n$). Making queries with $i = j$ is allowed.

The response to a comparison query is 0 if $a[i]$ "approximately equals" $a[j]$, otherwise: -1 if $a[i] < a[j]$, and 1 if $a[i] > a[j]$.

A swap query swaps values in $a[i]$ and $a[j]$, and the response to a swap query is 1 if after this swap the array became sorted in ascending order, and 0 otherwise.

Your program should exit as soon as it receives 1 as a response to a swap query.

The program should make at least one swap query. For example, if the hidden permutation is already sorted, it can make a query "`S 1 1`", receive a 1, and exit.

Examples

standard input	standard output
4 -1 -1 1 1	C 1 2 C 2 3 C 3 4 S 3 4
1 1	S 1 1

Problem 79. Gleb and Liteyny Avenue

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	512 megabytes

Gleb has just had a good time in a bar on Nekrasova street, and is now heading to the place of his accommodation that happened to be hotel Indigo. His path goes along Liteyny Avenue that we consider to be a perfect straight line.

For the purpose of this problem Gleb only moves along the avenue and across it. He now stays at point 0 along the avenue and the hotel he goes to is located at point L. Moreover, the hotel is located on the other side of the avenue. There are n pedestrian crossings between Gleb and Indigo, the i-th of them is located at integer point x_i. In order to get to the hotel Gleb needs to walk all the avenue from point 0 to point L and cross the street at some moment of this walk.

All crossings are equipped with a traffic light. All these traffic lights have cycles of the same length. They go green for g seconds and red for r seconds. However, they are not synchronized, i.e. each traffic light has some random equiprobable cycle shift. All shifts are integers, so there are only $r + g$ possible shifts in total. Gleb knows integers r and g but at beginning he knows nothing about the shifts.

Each crossing traffic light has a timer that shows the current state of this traffic light's cycle. If the light is green, the timer shows how many seconds are left to cross the street, if the light is red, the timer shows how many seconds to wait before the light goes green. Gleb is able to notice the light's color and the timer state only when he is at the exact point this crossing and traffic light are located. The good part is that Gleb is capable of remembering everything he has seen during this walk, so he keeps in mind the current state of all the traffic lights he has already seen.

All coordinates are integer and are given in meters. Gleb moves with a speed of one meter per second. Moreover, it is known that pedestrian crossings are not too frequent. Formally, there are no three crossings located within $g + r$ meters.

Gleb can move in any direction along the avenue and cross it arbitrary number of times. It takes b seconds to cross the avenue, and the traffic light must be green during all this period.

Input

The first line of the input contains integers n, L, g, r and b ($1 \le n \le 100\,000$, $1 \le L, g, r \le 10^9$, $1 \le b \le g$) — the number of crossings along the avenue, initial distance between Gleb and hotel Indigo in meters, the length of the green phase of the traffic light in seconds, the length of the red phase of the traffic light in seconds, the time it takes Gleb to cross the avenue.

Then follow n lines that describe crossings' positions $x_1, x_2, \ldots, x_n$ ($0 < x_1 < x_2 < \ldots < x_n < L$).

Output

Print one real value — the expected time it will take Gleb to get to Indigo in case he acts optimally. The absolute or relative error should not exceed 10^{-9}.

Examples

standard input	standard output
1 10 2 4 1 2	12.6666666666667
2 820 30 50 23 400 810	866.0250000000000
3 100 20 50 1 10 15 85	107.6141690962099